

Give 'std::string' a non-const '.data()' member function.

- Document number: P0272R0
- Date: 2016-02-13
- Reply-to: David Sankel david@stellarscience.com
- Audience: Library Evolution

Abstract

Is `std::string`'s lack of a non-const `.data()` member function an oversight or an intentional design based on pre-C++11 `std::string` semantics? In either case, this lack of functionality tempts developers to use unsafe alternatives in several legitimate scenarios. This paper argues for the addition of a non-const `.data()` member function for `std::string` to improve uniformity in the standard library and to help C++ developers write correct code.

Introduction

This paper brings to discussion an issue that was originally brought forward in [LWG issue 2391](#) by Michael Bradshaw. It was moved to a LEWG issue in 2015 and hasn't been looked at since. This is being elevated to a paper in the hopes that it will finally be addressed.

Use Cases

C libraries occasionally include routines that have `char *` parameters. One example is the `lpCommandLine` parameter of the `CreateProcess` function in the [Windows API](#). Because the `data()` member of `std::string` is `const`, it cannot be used to make `std::string` objects work with the `lpCommandLine` parameter. Developers are tempted to use `.front()` instead, as in the following example.

```
std::string programName;

// ...

if( CreateProcess( NULL, &programName.front(), /* etc. */ ) ) {
    // etc.
} else {
    // handle error
}
```

Note that when `programName` is empty, the `programName.front()` expression causes undefined behavior. A temporary empty C-string fixes the bug.

```
std::string programName;

// ...

if( !programName.empty() ) {

    char emptyString[] = {'\0'};

    if( CreateProcess( NULL, programName.empty() ? emptyString : &programName.front(), /* etc. */ ) ) {
        // etc.
    } else {
        // handle error
    }
}
```

If there were a non-const `.data()` member, as there is with `std::vector`, the correct code would be straightforward.

```
std::string programName;

// ...

if( !programName.empty() ) {

    char emptyString[] = {'\0'};
```

```

if( CreateProcess( NULL, programName.data(), /* etc. */ ) ) {
    // etc.
} else {
    // handle error
}
}

```

A non-const `.data()` `std::string` member function is also convenient when calling a C-library function that doesn't have const qualification on its C-string parameters. This is common in older codes and those that need to be portable with older C compilers.

Wording

The wording here was taken directly from [LWG issue 2391](#).

1. Change class template `basic_string` synopsis, [basic.string], as indicated:

```

namespace std {
    template,
    class Allocator = allocator >
    class basic_string {
    public:
        [...]
        // 21.4.7, string operations:
        const charT* c_str() const noexcept;
        const charT* data() const noexcept;
        charT* data() noexcept;
        allocator_type get_allocator() const noexcept;
        [...]
    };
}

```

2. Add the following sequence of paragraphs following [string.accessors] p3, as indicated:

charT* data() noexcept;

Returns: A pointer `p` such that `p + i == &operator` for each `i` in `[0, size())`.

Complexity: Constant time.

Requires: The program shall not alter the value stored at `p + size()`.